1 to 3

Corrected sentence:
"Set up a Git repository for a data science project and push it to GitHub to manage version control, track changes, and ensure code stability." ✅ Already well written!

Here's your **practical flow + exact commands (step-by-step)** so you can perform it easily in lab.

Practical 1: Set up a Git Repository and Push to GitHub

Step 1: Create project folder

Command:

mkdir data-science-project

Move inside folder:

cd data-science-project

Step 2: Initialize Git repository

Command:

git init

This creates a hidden .git folder to track changes.

Step 3: Create sample project files

Example:

Create README:

echo "# Data Science Project" > README.md

Create Python file:

echo "print('Hello Data Science')" > main.py

Step 4: Check repository status

Command:

git status

Shows untracked files.

Step 5: Add files to staging area

Command:

git add .

(Stages all files)

Step 6: Commit files

Command:

git commit -m "Initial commit"

This saves your first version.

Step 7: Create repository on GitHub

Go to GitHub → New Repository

Example name:

data-science-project

Do **NOT** initialize with README (important).

Click **Create Repository**

Step 8: Connect local repo to GitHub repo

Copy repo URL from GitHub, then run:

git remote add origin https://github.com/your-username/data-science-project.git

Check connection:

git remote -v

Step 9: Push project to GitHub

Command:

git branch -M main

Then:

git push -u origin main

Project uploaded successfully 🎉

Step 10: Verify upload

Refresh your GitHub repository page — files should appear.

Final Flow (for writing in practical file)

1. Create project directory
2. Initialize Git repository using git init
3. Create project files
4. Add files using git add .
5. Commit files using git commit -m "Initial commit"
6. Create repository on GitHub
7. Connect local repo using git remote add origin <repo_url>
8. Push files using git push -u origin main
9. Verify repository on GitHub

Corrected sentence:
"Create feature branches for the project, merge them, and resolve conflicts to maintain separate workspaces for team collaboration." ✅ Nicely written!

Here's your **step-by-step practical flow + commands (one by one)** exactly like you'd perform in lab.

Practical 2: Create Feature Branches, Merge Them, and Resolve Conflicts

Step 1: Check current branch

```
git branch
```

Usually shows:

```
* main
```

Step 2: Create a feature branch

Example: feature1

```
git branch feature1
```

Switch to it:

```
git checkout feature1
```

Shortcut (both together):

```
git checkout -b feature1
```

Step 3: Make changes in feature branch

Example edit file:

```
echo "Feature 1 update" >> main.py
```

Check status:

```
git status
```

Add changes:

```
git add .
```

Commit changes:

```
git commit -m "Added feature1 changes"
```

Step 4: Switch back to main branch

```
git checkout main
```

Step 5: Create another feature branch

git checkout -b feature2

Make changes:

echo "Feature 2 update" >> main.py

Add:

git add .

Commit:

git commit -m "Added feature2 changes"

Step 6: Merge feature branch into main

Switch to main:

git checkout main

Merge feature1:

git merge feature1

Merge feature2:

git merge feature2

Step 7: Resolve Merge Conflict (if conflict occurs)

Conflict example appears like:

<<<<<<< HEAD

Feature 1 update

=======

Feature 2 update

>>>>>>> feature2

Open file and manually edit:

Feature 1 update

Feature 2 update

Then save file.

Add resolved file:

git add main.py

Commit resolution:

git commit -m "Resolved merge conflict"

Step 8: Push merged changes to GitHub

git push origin main

Final Flow (for writing in practical record)

1. Check current branch using git branch
2. Create feature branch using git checkout -b feature1
3. Modify project files
4. Add changes using git add .
5. Commit changes using git commit -m "message"
6. Switch to main branch
7. Merge feature branch using git merge feature1
8. Resolve conflicts if present
9. Commit resolved files
10. Push updated project using git push origin main

Corrected sentence:
"Fork an open-source AI project, create a feature branch, submit a pull request, and address conflicts during the review process." ✅ Clear and exam-ready already.

Here's your **lab practical flow + exact commands (step-by-step)** so you can execute and also write it in your record.

Practical 3: Fork an Open-Source AI Project and Submit a Pull Request

Step 1: Fork the repository on GitHub

1. Open any open-source AI repository (example: a small ML repo).
2. Click **Fork** (top-right corner).
3. Repository gets copied to **your GitHub account**.

Example:

https://github.com/original-user/project-name

becomes

https://github.com/your-username/project-name

Step 2: Clone forked repository to local system

Copy your fork URL, then run:

```
git clone https://github.com/your-username/project-name.git
```

Move inside folder:

```
cd project-name
```

Step 3: Add original repository as upstream

This keeps your fork updated with the original project.

```
git remote add upstream https://github.com/original-user/project-name.git
```

Check remotes:

```
git remote -v
```

Output example:

```
origin
upstream
```

Step 4: Create a feature branch

```
git checkout -b feature-update
```

Now you're working safely without affecting the main branch 👍

Step 5: Make project changes

Example:

```
echo "New feature added" >> README.md
```

Check status:

```
git status
```

Add changes:

```
git add .
```

Commit changes:

```
git commit -m "Added new feature update"
```

Step 6: Push feature branch to GitHub fork

```
git push origin feature-update
```

Branch uploaded successfully 🚀

Step 7: Create Pull Request (PR)

On GitHub:

1. Open your forked repository
2. Click **Compare & Pull Request**
3. Add title + description
4. Click **Create Pull Request**

Now request goes to original repository owner.

Step 8: Handle merge conflicts (if reviewer requests changes)

First update your fork with latest upstream changes:

```
git fetch upstream
```

Switch to main branch:

```
git checkout main
```

Merge upstream updates:

git merge upstream/main

Go back to feature branch:

git checkout feature-update

Merge updated main:

git merge main

If conflict appears like:

<<<<<<< HEAD

your change

=======

original change

>>>>>>> upstream/main

Edit file manually → remove markers → keep correct content.

Then:

git add .

Commit resolved conflict:

git commit -m "Resolved merge conflict"

Push again:

git push origin feature-update

Pull request updates automatically ✅

Final Flow (for practical record submission)

1. Fork open-source AI repository from GitHub
2. Clone forked repository locally
3. Add upstream repository using git remote add upstream <url>
4. Create feature branch using git checkout -b feature-update
5. Modify project files
6. Stage changes using git add .
7. Commit changes using git commit -m "message"
8. Push branch using git push origin feature-update
9. Create pull request on GitHub
10. Resolve merge conflicts if required and update PR